

FINAL PROJECT DEFENSE

Library Management

Comprehensive implementation of Case 6: Book tracking, user history, and
algorithmic optimizations.

PROBLEM DESCRIPTION

Case 6: Managing Media Assets

Digital library systems require precise synchronization between physical inventory and user transaction logs.

- ✓ **Availability Logic:** Prevent borrowing of items already checked out by others.
- ✓ **Entity Integrity:** Users are prohibited from borrowing duplicate copies of the same book.
- ✓ **Data Lifecycle:** Maintain persistent records of all past borrow/return events.



SYSTEM DESIGN & DIRECTORY STRUCTURE

```
├── main.py          # CLI application entry point
├── data/            # JSON data files
│   ├── books.json
│   └── users.json
├── models/          # Data models
│   ├── book.py
│   ├── person.py
│   ├── user.py
│   └── __init__.py
├── services/        # Business logic
│   ├── library_service.py
│   └── __init__.py
├── utils/           # Utility functions
│   ├── files_handler.py  # JSON file I/O
│   └── __init__.py
└── tests/           # Unit tests
    ├── test_models.py
    ├── test_services.py
    ├── test_library.py
    └── __init__.py
```

Modular Architecture

The code is partitioned into logical layers to ensure maintainability and separation of concerns as per the **ITP2 Requirements**.

- ✓ **Models:** Clean data blueprints and attributes.
- ✓ **Services:** High-level business logic and rules.
- ✓ **Utils:** Low-level I/O and helper functions.
- ✓ **Data:** Structured JSON storage for persistence.

TEAM CONTRIBUTION BREAKDOWN



Assemgazy Olzhas : Data Engineer

Implemented JSON handling, file I/O operations, and hash-map optimization for $O(1)$ lookups.



Miras Tulkibay : OOP Architect

Designed class hierarchies, including Encapsulation of availability states and Inheritance models.



Kuan Amirlan : Logic Master

Developed `library_service.py` core rules, generators, and advanced functional filtering.



Mansur Saginbek : Integration Director

Orchestrated the `main.py` loop, implemented decorators, and conducted unit testing.

DATA PERSISTENCE & OPTIMIZATION

Assemgazy Olzhas : Storage Layer and Optimization

The system transforms JSON lists into dictionaries using unique IDs as keys. This ensures that book and user lookups do not slow down as the database grows, similar logic was used for custom command find book by author

- ✓ Structured JSON format for Catalog and Users.
- ✓ Exception handling for missing data files.
- ✓ Memory-efficient data mapping.

$O(1)$

OBJECT-ORIENTED PROGRAMMING MODELS



Encapsulation

Private attributes for `__is_available` to protect data integrity via setters/getters.



Inheritance

User class inherits from Person, reusing core identity properties.



Polymorphism

Overriding `get_role()` across different user types for flexible processing.

Architecture by Miras Tulikbay: Ensuring code reusability and strict data boundaries.

PYTHON FEATURES & LOGIC



Iterators & Generators

Implemented yield in catalog rendering to minimize memory footprint when displaying available books.

Requirement	Implementation Logic	Validation Check
Borrow Book	Check ID availability and update Book/User records.	book.get_availability() == True
Return Book	Restore availability and append to history tuple.	Verify ID in User's active set.
History Tracking	Maintain immutable tuples of past transactions.	Prevents data tampering.

Logic by Amirlan Kuan : Using listed features and implementing logic

| INTERFACE & TRANSACTION DECORATORS

Mansur Saginbek : UI & Observability

The main.py loop provides an intuitive terminal interface for users, wrapped with systemic monitoring.

- ✓ **@log_transaction:** A decorator that timestamps each borrow event automatically.
- ✓ **Main Loop:** Robust while True structure with safe exit paths.
- ✓ **Integrated Workflows:** Seamlessly bridges Models and Services.

```
[SYSTEM] Borrowing ID: 101  
[LOG] Time: 10:15 AM  
[INFO] Transaction Successful.  
Inventory Updated.
```


| APPLICATION OPERATIONAL WORKFLOW

